

PERFORMANCE AUDITOR

Especificação completa do produto

V1 funcional + evolução V2/V3/V4 + visão da versão final

Versão consolidada: 1.0 • Setembro de 2026

Objetivo: criar uma plataforma capaz de analisar a performance de websites, consolidar métricas do Lighthouse/PageSpeed Insights e evoluir para dados reais de usuários, IA, histórico, comparação, monitoramento e relatórios técnicos acionáveis.

Documento baseado nos requisitos e decisões registrados na conversa e no documento de requisitos V1.

1. Visão geral do produto

O Performance Auditor será uma ferramenta de auditoria técnica para desenvolvedores, equipes de e-commerce, agências e clientes. O usuário informa uma URL e escolhe o dispositivo de análise. O sistema executa uma auditoria, apresenta as principais métricas de performance, identifica oportunidades de melhoria e, na evolução completa, explica o porquê dos problemas, o que deve ser corrigido primeiro, como corrigir e se a correção realmente melhorou a performance.

Princípio do produto: não ser apenas um clone do PageSpeed Insights.

A proposta final é transformar dados técnicos em um plano de ação: “Meu site está ruim. Por quê? O que devo corrigir primeiro? Como corrijo? E a correção realmente melhorou a performance?”

1.1 Fontes de dados e motores

- Google PageSpeed Insights / Lighthouse: auditoria de laboratório.
- CrUX: dados reais de usuários, previstos a partir da V2.
- Análise de código: contextualização de causas e sugestões, especialmente na camada de IA.
- Performance Engine: normalização, classificação, scoring, priorização e comparação.
- IA/LLM: interpretação, causa provável, recomendações e exemplos de implementação.

2. Métricas e critérios de performance

A V1 deve trabalhar com as métricas disponíveis no resultado do Lighthouse/PageSpeed.

Métrica	Bom	Precisa melhorar	Ruim
LCP	$\leq 2,5$ s	2,5–4 s	> 4 s
INP	≤ 200 ms	200–500 ms	> 500 ms
CLS	$\leq 0,1$	0,1–0,25	$> 0,25$

Também devem ser apresentadas quando disponíveis: FCP, TTFB, Speed Index, Total Blocking Time, tamanho da página, quantidade de requests e indicadores relacionados a JavaScript, CSS e imagens.

Nota: os limites e integrações externas devem ser revisados conforme as versões atuais das APIs e das definições oficiais de Core Web Vitals antes da produção.

3. V1 — Auditoria funcional

A V1 representa o núcleo inicial do produto e deve entregar uma auditoria funcional de ponta a ponta.

Código	Requisito	Prioridade
RF-01	Permitir informar uma URL HTTP/HTTPS para análise.	Alta
RF-02	Permitir selecionar estratégia Mobile ou Desktop.	Alta
RF-03	Executar análise usando Google PageSpeed Insights/Lighthouse.	Alta
RF-04	Exibir Performance Score.	Alta
RF-05	Exibir LCP, CLS, INP, FCP e TTFB.	Alta
RF-06	Classificar métricas como Good, Needs Improvement ou Poor.	Alta
RF-07	Exibir principais audits do Lighthouse com título, descrição, score e valor.	Alta

Código	Requisito	Prioridade
RF-08	Informar URL final após redirecionamentos.	Média
RF-09	Exibir mensagens de erro para URL inválida ou falha na API.	Alta
RF-10	Disponibilizar endpoint de health check do backend.	Média

3.1 Requisitos funcionais detalhados

Código	Especificação
RF-11	Aceitar apenas URLs válidas iniciadas por http:// ou https:// e rejeitar entradas inválidas antes da análise.
RF-12	Indicar análise em andamento e impedir ações duplicadas durante o processamento.
RF-13	Extrair LCP, CLS, INP, FCP e TTFB do Lighthouse quando disponíveis.
RF-14	Normalizar métricas para representação consistente, incluindo unidade e status.
RF-15	Aplicar limites de Core Web Vitals e indicar visualmente o estado.
RF-16	Ordenar principais audits com score inferior a 1, priorizando menor score.
RF-17	Informar data/hora, URL solicitada, URL final e estratégia.
RF-18	Tratar falhas de rede, respostas inválidas, erros da API e entradas inválidas com mensagens compreensíveis.

3.2 Requisitos não funcionais da V1 e base do produto

Código	Requisito
RNF-01	Interface responsiva durante a análise, com estados de carregamento.
RNF-02	Frontend e backend desacoplados.
RNF-03	Chaves de API nunca expostas no frontend; segredos em variáveis de ambiente.
RNF-04	Validação independente no backend.
RNF-05	Observabilidade de erros sem exposição de segredos.
RNF-06	Falhas de serviços externos devem resultar em erro controlado.
RNF-07	TypeScript modular, tipado e organizado por responsabilidade.
RNF-08	Serviços de análise, normalização e classificação testáveis isoladamente.
RNF-09	Resultados claros e ordenados por impacto.
RNF-10	Suporte a desktop e dispositivos móveis.
RNF-11	Estrutura semântica e navegação adequada para acessibilidade.
RNF-12	Textos e unidades preparados para futura internacionalização.
RNF-13	Política clara de retenção para URLs/resultados armazenados.
RNF-14	Controle do uso de APIs externas e prevenção de análises duplicadas desnecessárias.

4. Critérios de aceite da V1

Código	Critério
CA-01	Uma URL válida inicia uma análise sem erro de validação.
CA-02	Uma URL inválida recebe mensagem clara sem chamada externa desnecessária.
CA-03	Mobile e Desktop são enviados como estratégias distintas.
CA-04	O resultado mostra score e métricas disponíveis.
CA-05	CLS é apresentado com precisão adequada e métricas temporais em milissegundos.
CA-06	Audits com problemas aparecem ordenados por severidade/score.
CA-07	Falhas do PageSpeed são tratadas de maneira compreensível.
CA-08	A chave da API, quando utilizada, não aparece no bundle do frontend.

4.1 Exemplo de experiência da V1

```
URL: https://meusite.com
Estratégia: Mobile / Desktop
```

```
Performance Score: 67
```

```
LCP 3.8s Poor
INP 180ms Good
CLS 0.31 Poor
FCP 2.1s Needs Improvement
TTFB 1.2s Needs Improvement
```

```
Problemas prioritários:
```

- CLS elevado
- LCP elevado
- Imagens sem otimização
- CSS bloqueando renderização

A V1 deve entregar dados técnicos confiáveis e bem apresentados. A interpretação profunda, recomendação contextualizada e histórico entram nas fases seguintes.

5. Arquitetura completa planejada

A arquitetura separa frontend e backend. O frontend coleta entradas e apresenta resultados; o backend concentra validação, integrações externas, normalização e regras de negócio.

Camada	Tecnologia / responsabilidade
Frontend	React + Vite + TypeScript; formulário, loading, cards de métricas, dashboard, histórico, gráficos e resultados.
Backend	Node.js + TypeScript + Express; API REST, validação, integração, normalização e regras de negócio.
Performance	Google PageSpeed Insights API / Lighthouse.
Dados de campo	CrUX API, previsto para V2.
Persistência	PostgreSQL + Prisma, previsto para V2.
IA	LLM para interpretação e geração de recomendações, previsto para V3.
Relatórios	HTML/PDF, evoluindo de relatório técnico para relatório executivo + técnico.
Produção	Docker, variáveis de ambiente, segurança, rate limiting, logs, testes, CI/CD e deploy.

5.1 Fluxo principal

1. Usuário acessa o sistema.
2. Informa uma URL.
3. Seleciona Mobile ou Desktop.
4. Frontend envia POST /api/analyze.
5. Backend valida a URL.
6. Backend consulta PageSpeed Insights/Lighthouse.
7. Backend normaliza score, métricas e audits.
8. Performance Engine classifica e prioriza.
9. Frontend apresenta resultado e oportunidades.
10. V2+ salva, compara e complementa com CrUX.
11. V3+ IA interpreta causas e recomendações.
12. V4+ monitoramento detecta regressões e dispara alertas.

5.2 Arquitetura lógica

```
FRONTEND React/TypeScript
|
v
BACKEND Node/TypeScript REST API
|
+---- PageSpeed / Lighthouse
|
+---- CrUX
|
v
Performance Engine
- Normalização
- Scoring
- Status
- Prioridades
- Comparações
|
+---- PostgreSQL / Prisma
|
+---- IA / LLM
|
v
Report Generator
- HTML
- PDF
```

6. Modelo de dados

Na V2, a entidade principal será Analysis. A evolução posterior adiciona Site/Project e AnalysisComparison.

Entidade	Principais campos
Site	id, name, url, createdAt, updatedAt
Analysis	id, siteId, url, strategy, score, analyzedAt, finalUrl
Metric	id, analysisId, name, value, unit, status
Audit	id, analysisId, auditId, title, description, score, displayValue
Comparison	id, baselineAnalysisId, currentAnalysisId, createdAt
Recommendation	id, analysisId, metric/audit, priority, title, description, suggestedFix

Evoluções futuras de persistência

- Histórico consultável por URL, data e estratégia.
- Projetos/sites para organizar múltiplas análises.
- Comparações antes/depois.
- Recomendações vinculadas à análise e ao problema.
- Retenção e privacidade configuráveis.

7. Dashboard e experiência do produto

O dashboard completo deve sair de uma simples tela de resultado e se tornar o centro operacional do produto.

7.1 Dashboard principal

- Score geral de performance.
- Scores/status de LCP, INP, CLS, FCP e TTFB.
- Resumo de problemas prioritários.
- Quantidade de problemas encontrados e quantidade de alto impacto.
- Ação para gerar plano de correção.
- Gráficos de evolução quando houver histórico.
- Metas de performance e indicação de cumprimento.

7.2 Exemplo visual conceitual

```
PERFORMANCE AUDITOR

Performance 67

LCP 3.8s Poor
INP 180ms Good
CLS 0.31 Poor
FCP 2.1s Needs Improvement
TTFB 1.2s Needs Improvement

Problemas prioritários
P0 CLS elevado
P0 LCP elevado
P1 Imagens sem otimização
P1 CSS bloqueando renderização

IA
5 problemas encontrados
2 possuem alto impacto

[ Gerar plano de correção ]
```

7.3 Auditor detalhado

- Título e descrição do audit.
- Score e valor.
- Métrica relacionada.
- Severidade/prioridade.
- Evidência técnica.
- Recomendação.
- Sugestão de implementação quando aplicável.

8. Inteligência Artificial

A IA é uma camada de interpretação. Ela não deve substituir a evidência do Lighthouse/PageSpeed/CrUX; deve transformar os resultados em explicações acionáveis.

8.1 Responsabilidades

- Interpretar métricas e audits.
- Agrupar problemas relacionados.
- Identificar causa provável com base nas evidências disponíveis.
- Priorizar problemas por impacto.
- Produzir recomendações técnicas contextualizadas.
- Estimar impacto esperado apenas quando houver base suficiente, deixando claro quando for estimativa.
- Sugerir implementação/código quando houver evidência suficiente.
- Gerar plano de ação.

8.2 Exemplo de diagnóstico

CLS — 0,31 — Impacto alto — Prioridade P0

Problema: elemento visual está sendo inserido sem espaço reservado.

Possível causa: banner/carrossel carregado após o conteúdo inicial.

Recomendação: definir width/height ou aspect-ratio no container.

Impacto esperado: alto.

8.3 Análise de código

```
<ProductBanner image={product.image} />
```

Possível causa de CLS:
o componente pode não reservar espaço para a imagem antes do carregamento.

Sugestão:

```
<div style={{ aspectRatio: '16 / 9' }}>
  <img
    src={product.image}
    width={1280}
    height={720}
    loading="lazy"
    alt=""
  />
</div>
```

A IA deve diferenciar claramente evidência, causa provável, recomendação e estimativa, evitando apresentar hipótese como fato.

9. Relatório profissional

O relatório final deverá deixar de ser apenas uma listagem de métricas e apresentar uma narrativa técnica e executiva.

9.1 Estrutura do relatório final

Seção	Conteúdo
Capa	Site analisado, data, estratégia, versão da análise.
Resumo executivo	Score, principais problemas, ganhos e prioridades.
Core Web Vitals	LCP, INP, CLS e status.
Métricas complementares	FCP, TTFB, Speed Index, TBT e outros dados disponíveis.
Dados de campo	CrUX, quando disponível.
Lab x Field	Comparação entre laboratório e experiência real.

Seção	Conteúdo
Problemas	Audits e problemas agrupados por impacto.
Causa provável	Interpretação técnica baseada nas evidências.
Recomendação	Ação sugerida e prioridade.
Evidência	Dados que sustentam o diagnóstico.
Código	Exemplo de implementação quando seguro/aplicável.
Antes/depois	Comparação de análises e evolução das métricas.
Plano de ação	Lista priorizada para execução.

9.2 Exemplo de antes/depois

Métrica	Antes	Depois	Evolução
LCP	3,8 s	2,1 s	↓ 44%
CLS	0,31	0,06	↓ 81%
INP	280 ms	145 ms	↓ 48%

Resumo da otimização: foram identificados problemas de performance e, após as correções, as métricas devem ser comparadas para comprovar a evolução. O relatório deve destacar os principais ganhos e relacioná-los às alterações realizadas.

10. Histórico, comparação e monitoramento

Esses recursos transformam o auditor em uma ferramenta contínua de acompanhamento.

- Histórico: salvar análises e permitir consulta por URL, data e estratégia.
- Comparação: comparar baseline e análise atual e calcular evolução/regressão.
- Projetos: agrupar sites e análises recorrentes.
- Monitoramento: executar análises programadas.
- Regressão: identificar deterioração de métricas.
- Alertas: notificar quando uma métrica ultrapassar limite ou sofrer regressão relevante.
- Metas: definir objetivos de performance por projeto/site.

11. Roadmap completo — V1 → versão final

Versão	Objetivo	Entregas principais
V1	Auditoria funcional	URL, Mobile/Desktop, PageSpeed, Lighthouse, métricas, status e audits.
V2	Histórico e evidências	PostgreSQL, Prisma, histórico, CrUX, comparação antes/depois e relatório.
V3	Inteligência	IA para causa provável, prioridade, recomendações e sugestões de código.
V4	Produto	Projetos, monitoramento agendado, regressões, alertas e relatórios recorrentes.
Final	Plataforma profissional	Integração de todas as capacidades, segurança, observabilidade, testes, CI/CD, Docker, d

11.1 Fases de implementação

Fase 1 — Core

- React + TypeScript.
- Node + TypeScript.
- PageSpeed API.
- Lighthouse.
- Métricas, audits e scoring.

Fase 2 — Banco

- PostgreSQL + Prisma.
- Usuários, projetos, sites, análises, métricas, audits e recomendações.

Fase 3 — Dashboard

- Dashboard principal, histórico, gráficos, detalhes, comparação e filtros.

Fase 4 — CrUX

- Dados reais, Core Web Vitals reais, Lab × Field e histórico de campo.

Fase 5 — IA

- Interpretação, causa provável, prioridade, impacto, recomendações e implementação.

Fase 6 — Relatórios

- Relatório executivo, relatório técnico, HTML, PDF, antes/depois e plano de ação.

Fase 7 — Monitoramento

- Análises agendadas, histórico automático, regressão, alertas e metas.

Fase 8 — Produção

- Docker, variáveis de ambiente, segurança, rate limiting, logs, testes, CI/CD, documentação da API e deploy.

12. Segurança, confiabilidade e qualidade

12.1 Segurança

- Segredos e chaves somente no backend/variáveis de ambiente.
- Validação de entrada no frontend e backend.
- Rate limiting para proteger a API.

- Tratamento seguro de erros sem exposição de segredos.
- Política de retenção para URLs e resultados.

12.2 Confiabilidade

- Falhas do PageSpeed/Lighthouse devem resultar em erro controlado.
- Respostas inválidas e falhas de rede devem ser tratadas.
- Análises duplicadas desnecessárias devem ser evitadas.
- Estados de loading e erro devem ser claros.

12.3 Testes

- Testes unitários do serviço de análise.
- Testes de normalização.
- Testes das regras de classificação.
- Testes de validação de URL.
- Testes de integração da API.
- Testes dos fluxos principais do frontend.

13. API e organização sugerida

A API deve manter responsabilidades separadas e permitir evolução sem acoplamento excessivo.

```
POST /api/analyze
GET /api/health

Evolução:
GET /api/sites
POST /api/sites
GET /api/sites/:id/analyses
GET /api/analyses/:id
POST /api/analyses/:id/compare
POST /api/reports
POST /api/monitoring
GET /api/alerts
```

13.1 Estrutura conceitual de projeto

```
performance-auditor/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├── hooks/
│   │   ├── types/
│   │   └── ...
│   └── backend/
│       ├── src/
│       │   ├── controllers/
│       │   ├── routes/
│       │   ├── services/
│       │   ├── pagespeed/
│       │   ├── lighthouse/
│       │   ├── performance/
│       │   ├── crux/
│       │   ├── ai/
│       │   ├── reports/
│       │   ├── validators/
│       │   ├── middleware/
│       │   ├── types/
│       │   └── ...
│       └── prisma/
```

```
■■ docker/  
■■ tests/  
■■ README.md
```

14. Comportamento esperado da versão final

A versão final deve entregar o seguinte fluxo completo:

1. Usuário cadastra ou informa um site.
2. Escolhe estratégia Mobile/Desktop.
3. Executa uma auditoria.
4. Sistema coleta PageSpeed/Lighthouse.
5. Sistema consulta CrUX quando disponível.
6. Performance Engine normaliza e classifica.
7. Dashboard mostra score e métricas.
8. Sistema agrupa problemas.
9. IA identifica causas prováveis com base nas evidências.
10. IA gera recomendações e, quando possível, exemplos de código.
11. Usuário registra/consulta histórico.
12. Usuário compara baseline x atual.
13. Sistema gera relatório HTML/PDF.
14. Monitoramento executa análises recorrentes.
15. Sistema detecta regressões.
16. Sistema dispara alertas.
17. Relatório recorrente comprova evolução.

14.1 Pergunta que o produto deve responder

“Meu site está ruim. Por quê? O que devo corrigir primeiro? Como corrijo? E a correção realmente melhorou a performance?”

14.2 Resultado ideal de uma análise

Performance: 67/100

CLS — 0.31 — Poor

Impacto: ALTO

Prioridade: P0

Problema:

Elemento visual está sendo inserido sem espaço reservado.

Possível causa:

Banner/carrossel carregado após o conteúdo inicial.

Recomendação:

Definir width/height ou aspect-ratio no container.

Evidência:

Audit/Lighthouse + elemento identificado.

Sugestão de implementação:

Reservar dimensão antes do carregamento.

Depois da correção:

CLS 0.31 → 0.04

Melhoria: 87%

15. Critérios de sucesso do produto completo

- A auditoria V1 funciona de ponta a ponta para URLs válidas.
- Mobile e Desktop são tratados como estratégias independentes.
- Métricas e audits são normalizados e classificados.
- O sistema mantém histórico confiável.
- CrUX complementa os dados de laboratório quando disponível.
- Comparações mostram evolução e regressão.
- A IA transforma evidências em diagnóstico e plano de ação.
- Sugestões de código são fornecidas apenas quando houver evidência suficiente.

- Relatórios HTML/PDF são profissionais e acionáveis.
- Monitoramento recorrente identifica regressões.
- Alertas informam mudanças relevantes.
- Segurança, privacidade, testes e observabilidade estão incorporados.
- A aplicação é responsiva, acessível e preparada para internacionalização.
- O projeto pode ser executado e implantado de forma reproduzível com Docker/CI/CD.

16. Escopo consolidado e observações

Este documento consolida os requisitos presentes na conversa e no documento “Performance Auditor — Documento de Requisitos — V1 e Roadmap do Produto”. A separação V1/V2/V3/V4 foi preservada, enquanto a visão da versão final integra todas as capacidades descritas.

Não foram adicionados requisitos funcionais externos que não estejam sustentados pelo material fornecido. Onde a conversa tratou uma capacidade como futura, ela permanece identificada como evolução.

Antes de produção, integrações externas, limites de Core Web Vitals e detalhes das APIs devem ser revalidados contra suas versões oficiais atuais.

Fim do documento.

PERFORMANCE AUDITOR

Atualização da especificação — WebPageTest API

Versão do documento: 1.1 • Setembro de 2026

Esta seção complementa o documento consolidado original, adicionando o WebPageTest como motor de análise técnica avançada e atualizando arquitetura, fluxo, modelo de dados e roadmap.

1. WebPageTest API

O WebPageTest será uma fonte complementar de análise profunda. O PageSpeed Insights/Lighthouse permanece como o principal motor de auditoria de laboratório, enquanto o CrUX representa dados de experiência de usuários reais. O WebPageTest acrescenta uma visão detalhada do carregamento da página, waterfall, requests, recursos e métricas do teste.

1.1 Papel no produto

A integração deverá ser usada para responder perguntas que exigem evidência técnica mais detalhada, por exemplo: quais requests estão consumindo mais tempo, quais recursos estão aumentando o peso da página, quais terceiros participam do carregamento e onde está o gargalo observado no waterfall.

1.2 Capacidades planejadas

- Executar testes automatizados de uma URL a partir do backend.
- Coletar métricas de carregamento e desempenho retornadas pelo teste.
- Analisar waterfall e requests individuais.
- Identificar volume transferido e distribuição dos recursos.
- Agrupar recursos por JavaScript, CSS, imagens, fontes e terceiros.
- Complementar a análise de Core Web Vitals quando os dados estiverem disponíveis.
- Registrar o identificador do teste e o timestamp para histórico e comparação.
- Usar WebPageTest como investigação avançada quando PageSpeed/Lighthouse detectar problemas relevantes.

1.3 Importante sobre acesso e custo

O WebPageTest possui um plano Starter gratuito para execução de testes no site, mas a documentação oficial informa que a WebPageTest API é um recurso do plano Pro. Portanto, a automação por API não deve ser tratada no projeto como uma API gratuita e ilimitada. A aplicação deve encapsular a integração, controlar quota/uso e permitir que a funcionalidade seja habilitada somente quando houver uma chave válida.

Para o volume planejado de aproximadamente 20–30 URLs por dia, a integração continua tecnicamente viável, mas deve considerar a quantidade de test runs, principalmente se forem executadas múltiplas cargas por URL ou testes Mobile/Desktop.

1.4 Dados que interessam ao Performance Auditor

- Métricas de carregamento e desempenho do teste.
- Core Web Vitals disponíveis no resultado.
- Waterfall e evidências por request.
- Quantidade de requests e bytes transferidos.
- Distribuição dos recursos por tipo.
- Informações de terceiros e domínios envolvidos no carregamento.
- Dados CrUX disponibilizados pelo WebPageTest quando presentes.
- Referência do teste, status e data/hora.

2. Integração arquitetural

A arquitetura passa a ter três fontes principais de performance: PageSpeed/Lighthouse para laboratório, CrUX para campo e WebPageTest para investigação técnica avançada.

Fonte	Papel	Uso no produto
PageSpeed / Lighthouse	Laboratório	Score, métricas, audits e oportunidades.
CrUX	Usuários reais	Core Web Vitals e experiência real quando disponível.
WebPageTest API	Análise profunda	Waterfall, requests, recursos, métricas do teste e evidências técnicas.

Fluxo recomendado

POST /api/analyze → validação → PageSpeed/Lighthouse → normalização → Performance Engine → regra de severidade → WebPageTest API (quando necessário) → normalização WPT → consolidação → dashboard/relatório.

3. Requisitos funcionais adicionados

- RF-20 — Permitir executar WebPageTest como análise técnica complementar.
- RF-21 — Normalizar métricas e evidências retornadas pelo WebPageTest.
- RF-22 — Registrar o testId e o status do teste quando disponíveis.
- RF-23 — Permitir execução sob demanda ou por regra de severidade.
- RF-24 — Evitar análises WebPageTest duplicadas desnecessárias.
- RF-25 — Tratar timeout, falha de fila, erro de autenticação e resposta inválida do serviço.
- RF-26 — Manter a chave do WebPageTest somente no backend/variáveis de ambiente.

4. Modelo de dados atualizado

Entidade	Campos adicionais sugeridos
Metric	source, name, value, unit, status
Audit	source, auditId, title, description, score, displayValue
WebPageTestResult	id, analysisId, testId, status, metrics, requests, bytes, waterfallRef, analyzedAt
Analysis	webPageTestStatus, webPageTestTestId, webPageTestAnalyzedAt

A recomendação é manter source explícito nas métricas e audits. Isso permite diferenciar, por exemplo, uma métrica de laboratório do Lighthouse, uma métrica de campo do CrUX e uma métrica técnica do WebPageTest.

5. Estratégia de execução

O WebPageTest não precisa obrigatoriamente ser executado em todas as análises. Para reduzir tempo, consumo e dependência externa, o Performance Auditor pode adotar uma estratégia em duas etapas:

- Etapa 1 — PageSpeed/Lighthouse + CrUX: coleta inicial e classificação.
- Etapa 2 — WebPageTest: executado quando solicitado pelo usuário ou quando regras de severidade indicarem necessidade de diagnóstico profundo.

Exemplos de gatilhos para análise avançada:

- LCP ou TTFB acima do limite definido.
- Grande quantidade de requests.
- Peso elevado de JavaScript, CSS, imagens ou fontes.
- Problemas relevantes de renderização ou carregamento identificados pelo Lighthouse.
- Necessidade de waterfall para investigar a causa de um problema.
- Comparação antes/depois em uma otimização importante.

6. Roadmap atualizado

Fase	Entrega
V1	PageSpeed/Lighthouse, métricas, status e audits.

Fase	Entrega
V2	Histórico, PostgreSQL/Prisma, CrUX, comparação e WebPageTest como análise avançada.
V3	IA para interpretar evidências de PageSpeed, CrUX e WebPageTest.
V4	Monitoramento, regressões, alertas e relatórios recorrentes.
Final	Integração completa das três fontes, histórico, IA, relatórios, observabilidade, segurança e CI/CD.

7. Exemplo de resultado consolidado

Performance: 67/100

LCP — 3,8 s — Poor

CLS — 0,31 — Poor

CrUX — experiência real com LCP acima do objetivo

WebPageTest — waterfall identifica request de imagem/banner como um dos maiores tempos de carregamento

Impacto: ALTO

Prioridade: P0

Diagnóstico: o laboratório aponta o problema; o CrUX mostra sua ocorrência em usuários reais; o WebPageTest fornece evidências para localizar o gargalo técnico.

8. Referência oficial utilizada

A documentação oficial do WebPageTest informa que o Starter é gratuito para testes e que a WebPageTest API faz parte do plano Pro. Também informa que a API fornece métricas de Core Web Vitals e pode incluir dados específicos de URL do CrUX nos resultados. Essas condições devem ser revalidadas antes da produção, pois planos e limites podem mudar.

Nota: esta atualização corrige a interpretação de que a WebPageTest API seria necessariamente gratuita. O uso gratuito do WebPageTest não implica acesso gratuito e ilimitado à API.